

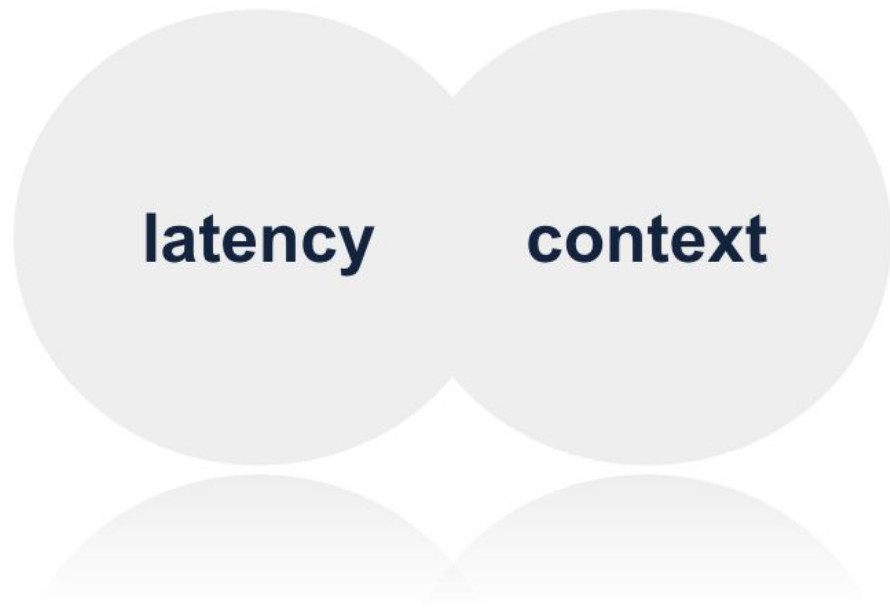
REAL-TIME FRAUD DECISION ENGINE

online FDS

Project Architecture & Full Flow Document

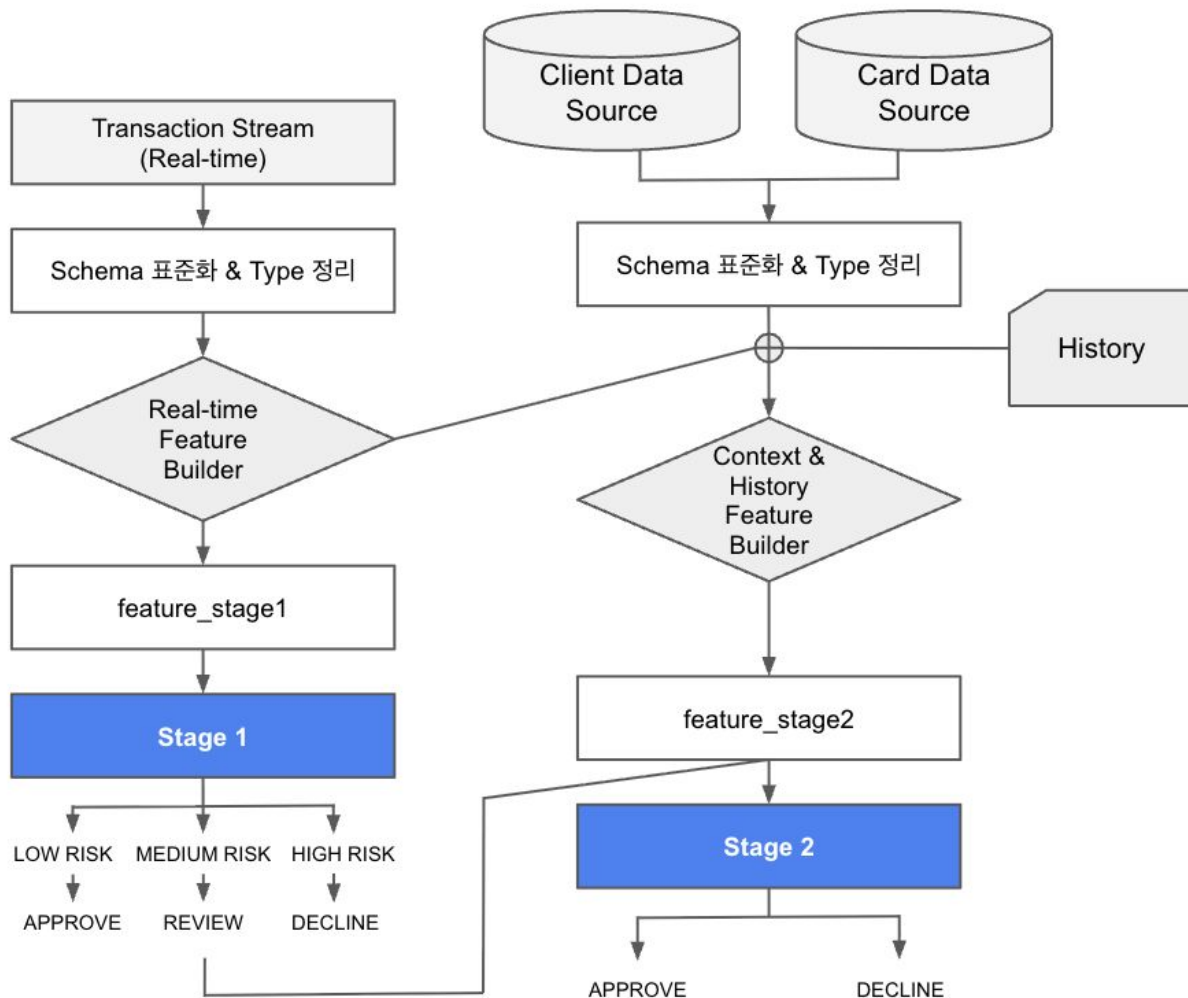
Team4 김나경 김채현 김형준 정준우

Project Definition — Why



이 프로젝트가 해결하는 문제의 본질

- 온라인 카드 거래는 실시간 판별이 필요하다
- 하지만 사기 탐지는 맥락(context)을 요구한다
- **Latency vs Context**는 충돌한다
- 이 충돌을 설계적으로 해결하고 싶었다



Offline

- 데이터를 읽고 구조를 분석
- Stage1/Stage2 피쳐 설계
- 모델을 학습 + drift 대응
- Threshold 결정
- 모델과 설정을 artifact로 저장

Online

- 실시간 거래가 들어옴
- 동일한 피쳐 생성 로직 실행
- 저장된 모델로 score 계산
- 저장된 threshold로 의사결정
- 결과 즉시 반환

Data Structure

Dataset

CSV Files (3)

transactions_data.csv

Transaction-level records

- Amount, timestamp, merchant info
- Transaction type
- Core unit for fraud analysis

cards_data.csv

Card-level information

- Credit / debit card type
- Card limits
- Activation date
- Linked via card_id

users_data.csv

User-level information

- Demographics
- Account-related attributes

JSON Files (2)

mcc_codes.json

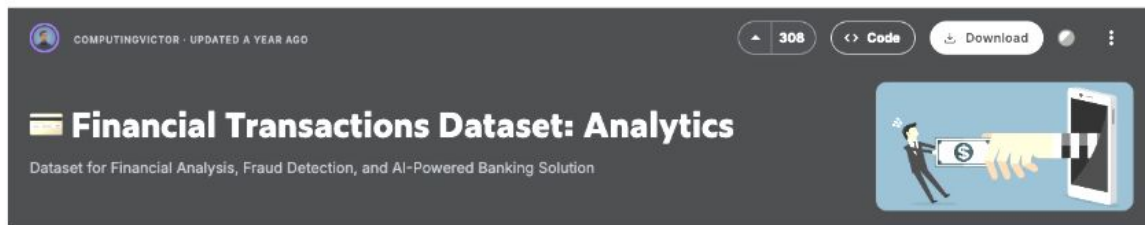
Merchant Category Codes

- Industry-standard MCC classification
- Business type descriptions

train_fraud_labels.json

Fraud labels

- Binary indicator (fraud / non-fraud)
- Used for supervised learning



Target Imbalance

Train 기준 positive rate $\approx 1.08\%$ (모든 실험에서 Recall ≈ 0.70 고정)

Sampling 전략	Class Weight
Raw	No
Under	No
Over	No
Raw	Yes
Under	Yes
Over	Yes



전략	PR-AUC	Precision @Recall	Alert Rate
raw + no_cw	0.2568	0.0986	0.1303
under + no_cw	0.2550	0.0990	0.1298
over + no_cw	0.2458	0.1026	0.1253
raw + cw	0.2362	0.1049	0.1225
under + cw	0.2347	0.1049	0.1226
over + cw	0.2244	0.1051	0.1223

raw + no_cw

- 실제 운영 환경 분포를 그대로 반영
- 가장 높은 PR-AUC 확보
- threshold 안정적 (0.016 수준)
- calibration 왜곡 최소화
- drift 대응 시 해석 및 관리 용이

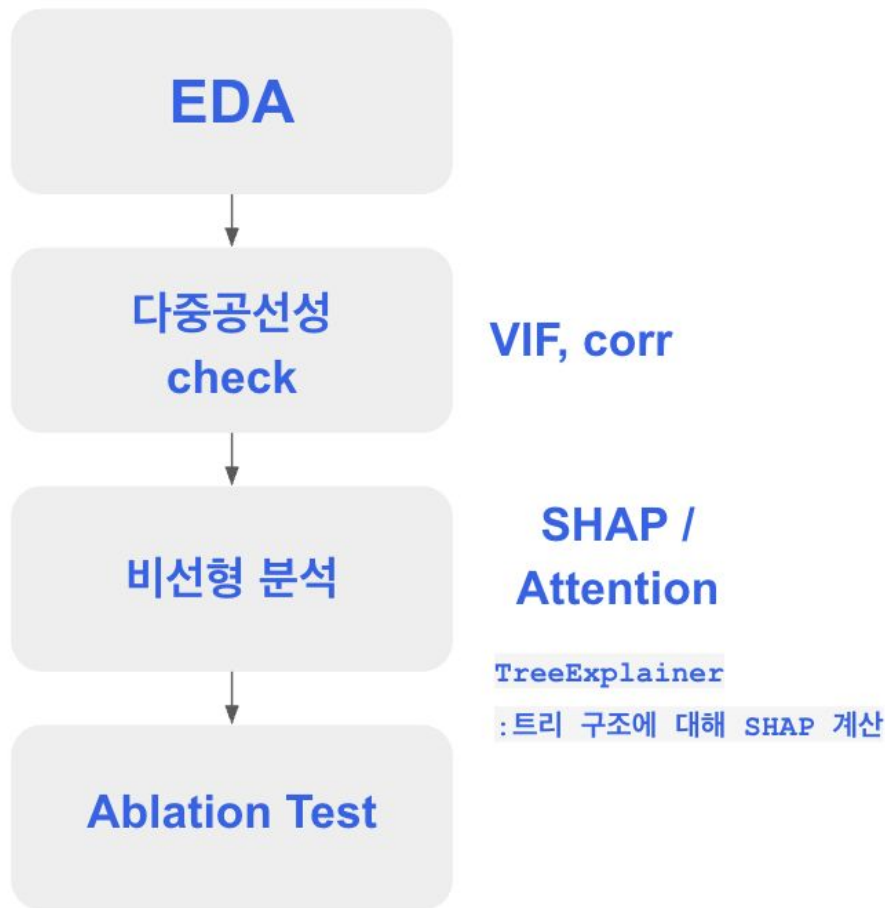
EDA & FEATURE

데이터 정제

- 2011년 제거 (라벨 이상치)
- 온라인 거래만 필터링
- 단일값 컬럼 제거
- 극단값 구간 식별 및 별도 처리

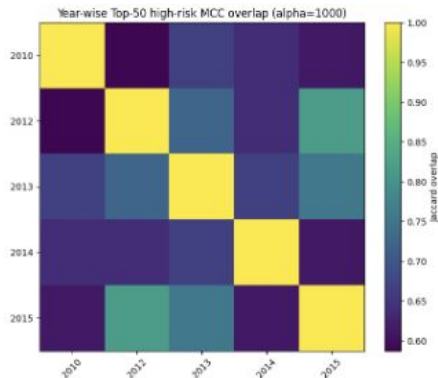
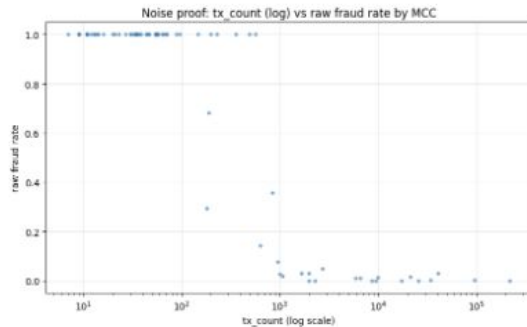
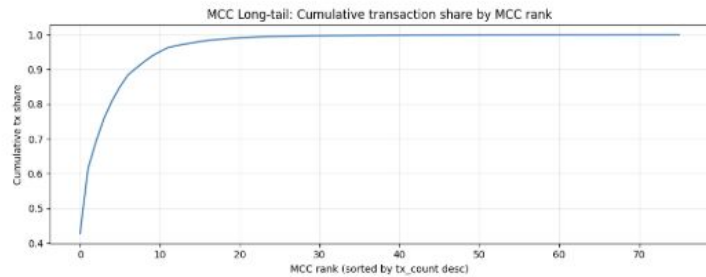
피처 파생

- 로그 변환 (heavy-tail 완화)
- Cyclic encoding (시간 주기 보존)
- Bayesian smoothing (MCC 위험도 안정화, $\alpha=1000$)
- Shift 기반 이력 생성 (leakage 방지)
- Rolling 집계 피처
- 고위험 구간 이진 flag
- Velocity $\times$ Novelty $\times$ Risk interaction 설계



1. MCC Bayesian Smoothing - long tail noise 문제와 해결

가설: 업종(MCC)은 강한 신호다 — 단, 그대로 쓰면 노이즈가 된다



롱테일 구조 확인

- 상위 5~10개 MCC가 전체 거래의 90% 차지
- 나머지 MCC는 거래량 극소 → raw fraud rate = 1.0인 MCC 다수 존재

tx_count vs raw fraud rate 산점도

- 거래량 적은 MCC에서 rate=1.0 다수
- 노이즈 입증

연도별 jaccard 행렬 (안정성 입증)

Bayesian smoothing 적용 → raw vs smoothed 비교 scatter → 저표본은 평균으로 수축, 고표본은 원래 rate 유지 확인

α 민감도 분석(100/500/1000/2000) → $\alpha=1000$ 채택

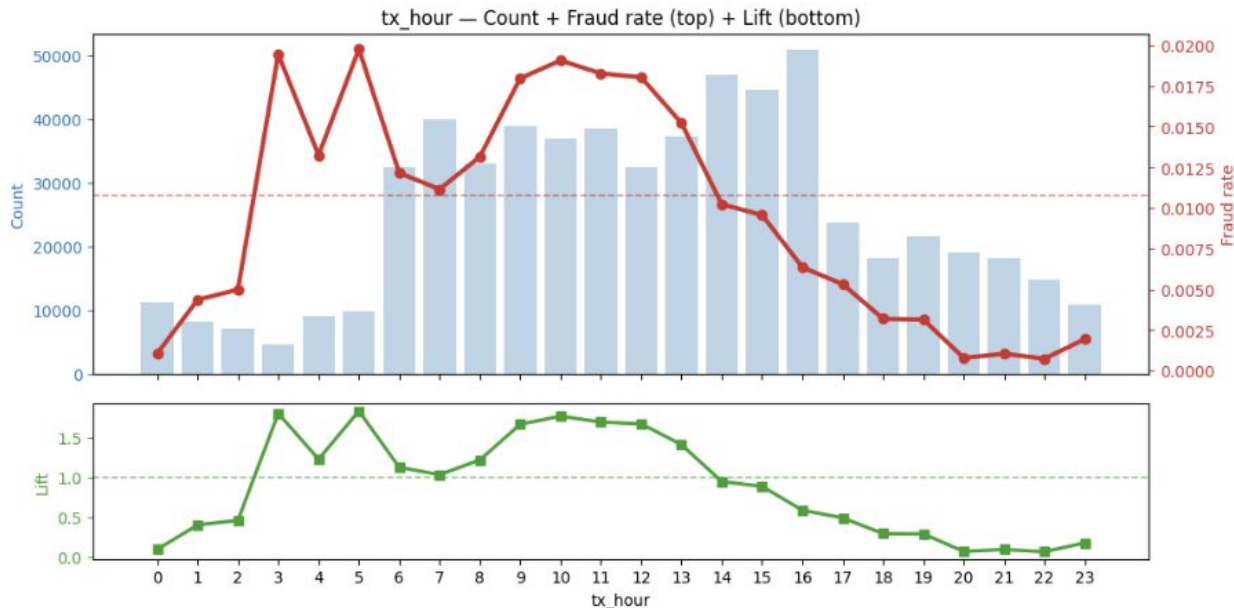
연도별 Jaccard Overlap 0.6~0.85 → 구조적 안정성 입증

MCC 종류 자체로 강한 신호
→ Stage1에서 사용

Baseline 대비 Δ PR +0.553, Δ Lift +4.49

2. Cyclic Encoding — raw hour의 구조적 한계

가설: 사기는 특정 시간대에 집중된다. 단 raw hour는 23시와 0시를 멀다고 인식해 주기 구조를 담지 못한다.

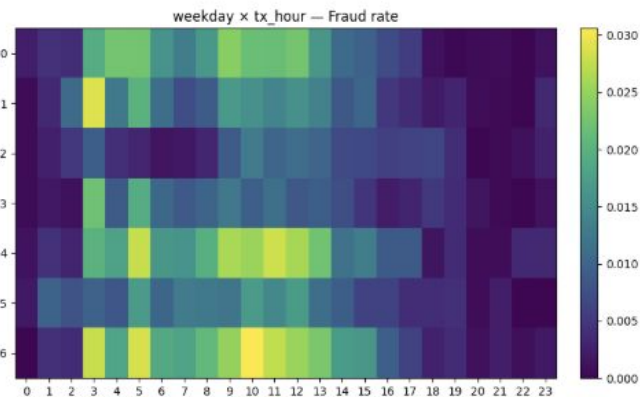
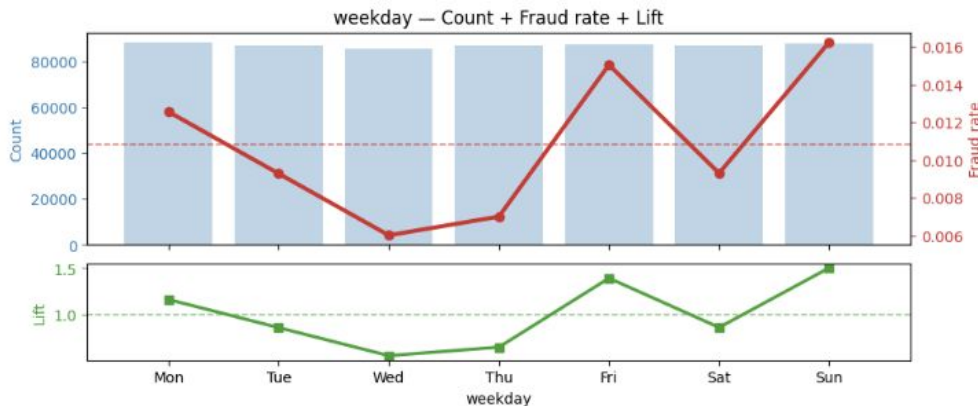


=> hour 특징은 sin + cos feature로 사용

1. 새벽 3~5시(Lift 1.81), 오전 10~12시(Lift 1.76) spike 확인
2. raw hour vs sin+cos 성능 비교
→ 29% 개선
3. Δ Lift 비교
→ hour_cos +0.337,
hour_sin +0.295
vs raw hour +0.127
4. VIF 확인
→ sin은 raw와 겹치지만(VIF 3.09)
cos는 독립(VIF 1.00)
→ 다중공선성 없음

3. Weekday × Hour Interaction — 단독보다 조합이 강하다

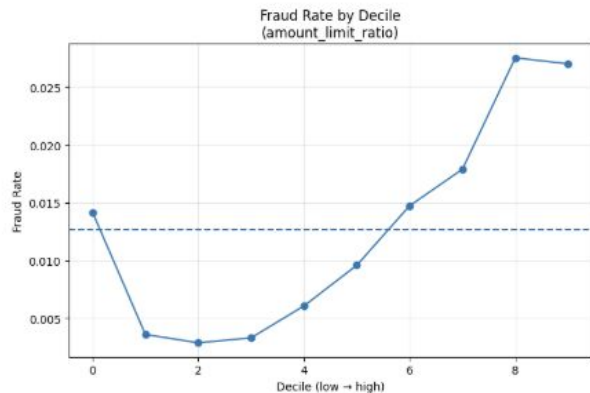
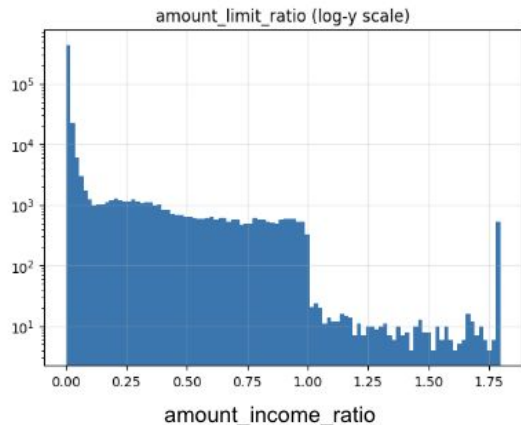
가설: 주말 새벽과 평일 새벽의 리스크는 다르다. hour 단독이 아닌 요일×시간 조합이 실제 사기 패턴을 포착한다.



1. weekday 단독 Lift
→ 월/금/일 구간 약한 상승 확인 (Δ Lift +0.081)
2. weekday × hour 히트맵 관찰
3. 최고 조합 일요일 10시 Lift **2.83** 발견
4. 상위 2개 조합으로 `is_risky_wd_hour` 생성
(2개는 조합 수를 바꿔가며 관찰한 결과)
5. `is_risky_wd_hour=1` fraud rate 2.74%
→ Lift **2.54×**

4. amount_limit_ratio 극단값 — clip하면 오히려 나빠진다

가설: 사기범은 한도를 거의 소진하는 방식으로 인출한다. 극단값이 노이즈가 아니라 핵심 신호다.



1. 단일 피처 성능

PR-AUC = **0.071** (단일 피처 최고)

Add-one Δ Lift = **+0.329** (전체 1위)

→ Baseline 위에서도 강력한 추가 분리력 확보

2. 분포 구조

Extreme right-skew + heavy tail

대부분 거래는 한도의 1% 미만 사용

상위 0.1% 구간 fraud rate = **66.5%**

→ 극단값 자체가 핵심 사기 신호

3. 극단값 처리 실험

처리 방법	Δ PR_AUC	Δ Lift
원본 ratio	+0.036	+0.329
원본 + extreme_flag + low_credit_flag	+0.056	+0.199
clip ratio (q0.999)	+0.059	+0.008

clip 처리 시 PR_AUC는 높아지지만

Top-decile Lift가 크게 하락

극단값 자체가 fraud 신호의 핵심

연속형 유지 +
limit_ratio_extreme
이진 flag 추가 생성

Set Dataset for Stage 1 & 2

Stage 1 — Real-Time Lightweight Layer (Transaction-Only)

설계 원칙

- 실시간 추론 가능 (저지연)
- 거래 단위 정보만 활용
- 즉각적인 위험 신호 감지에 집중

Stage 2 — Contextual Deep Risk Layer

설계 원칙

- 고객·카드·가맹점 맥락 정보 반영
- 누적 이력 기반 위험 강화
- Stage 1 통과 거래에 대해 정밀 판별

Stage 1은 “즉각적인 위험 신호 감지 레이어”

→ 시간·금액·MCC·오류 기반의 빠른 필터링

Stage 2는 “맥락 기반 정밀 판단 레이어”

→ 이력·속도·신규성·한도 압박·행동 변화까지 반영

두 Layer는

속도(Real-Time Constraint)와

정확도(Contextual Depth)라는

서로 다른 제약을 해결하기 위한 구조적 설계

Model Selection as a Continuation of the Two-Core Constraints

Stage1

선택 기준

- 1) Latency 추론 시간이 짧아야 한다 (실시간 처리)
- 2) Recall 우선 부정거래를 최대한 Stage2로 넘겨야 한다

모델	Precision	Recall	F1	pred_sec
logit	0.834	0.455	0.589	0.037s
logit_calib	0.835	0.455	0.589	0.190s
hgb	0.822	0.448	0.580	0.114s
lgb_small	0.809	0.441	0.571	0.040s
gnb	0.517	0.293	0.374	0.027s

탈락 이유

- **logit_calib**
— 성능 동일, pred_sec 5배 증가. 보정 이점 없음
- **hgb / lgb_small**
— recall·precision 동시 열세. 트리 구조의 이점이 이 피쳐셋에서 발현되지 않음
- **gnb**
— recall 0.293. 운영 불가 수준

결론

Recall 최상위 + pred_sec 0.037s
Logistic Regression 채택

거래 단독 피쳐는 선형 분리 가능성이 높고,
연산이 가볍다

Model Selection as a Continuation of the Two-Core Constraints

Stage2

선택 기준

1) Latency 완화

허용 범위 내에서 성능 우선

2) Precision 강화

부정거래를 최대한 Stage2로 넘겨야 한다

3) F1 균형

Recall과 Precision의 균형

모델	Precision	Recall	F1	lat_p50	lat_p95
rf	0.989	0.956	0.972	259ms	291ms
hgb	0.994	0.947	0.970	1.95ms	2.04ms
lgb_small	0.995	0.945	0.969	0.96ms	1.02ms
gnb	0.881	1.000	0.937	0.46ms	0.47ms
logit	0.881	1.000	0.937	0.59ms	0.62ms

RF 미채택 이유

- F1 0.972
 - lgb_small 대비 미세 우위 (0.003차)
 - lat_p50 259ms
 - lgb_small 대비 270배 느림
 - 학습 시간 45s / 모델 크기 큼
 - / scale-out 비용 증가
- 성능 미세 우위가
운영 비용 열세를 정당화하지 못함

Model Selection as a Continuation of the Two-Core Constraints

Stage2

최종 성능 (튜닝 후, Test set)

	Precision	Recall	F1
fraud (class=1)	0.987	0.955	0.971
non-fraud (class=0)	0.730	0.910	0.810
Accuracy			0.949

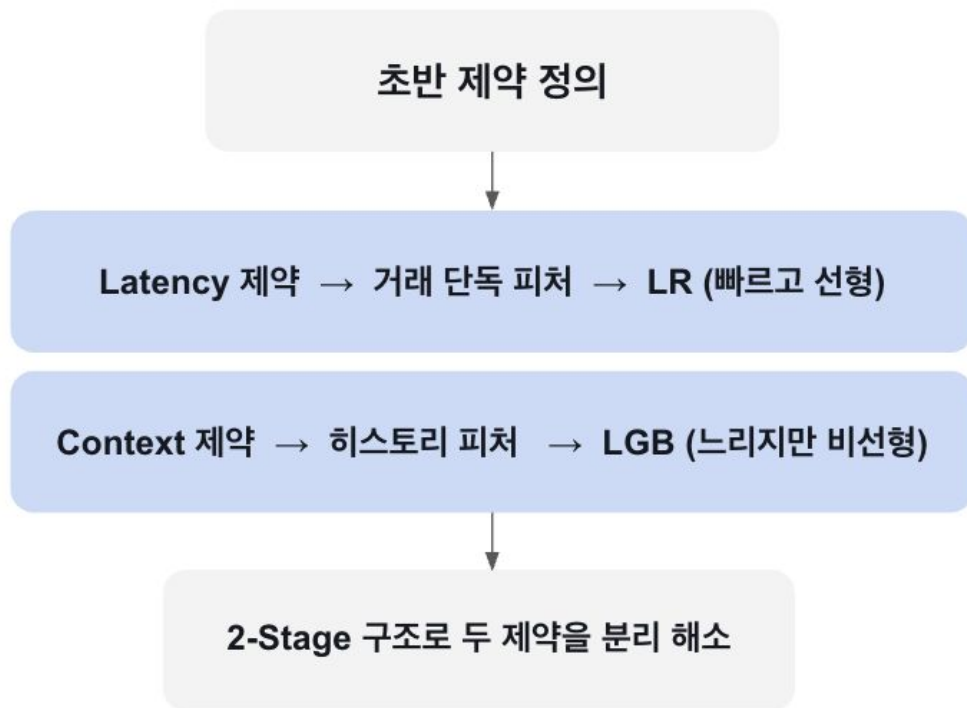
결론

- RF 수준 F1 유지 + latency 270배 단축
- 히스토리 피처의 복잡한 비선형 상호작용 포착
→ 트리 기반 필수
- lgb_small이 성능과 운영 효율의 균형점

Wrap up

프로젝트 전체 흐름으로 연결

	Stage1	Stage2
핵심 제약	Latency	Context
피처	거래 단독 (10개)	히스토리 포함 (20개)
모델	Logistic Regression	LightGBM
이유	선형 분리 가능 + 연산 가벼움	비선형 상호작용 포착
핵심 지표	Recall + pred_sec	F1 + lat_p95
Recall	0.455	0.955
Precision	0.834	0.987
추론 시간	0.037s	~1ms



모델이 미래 환경에서도 작동하는가 — Covariate Shift 점검

CHECK = 라벨 없는 미래 환경 데이터 (166,523건)

Train = 608,430건

라벨 없으므로 성능 직접 측정 불가

→ 입력 분포 변화(drift)로 간접 판단

주요 발견

피처	PSI	KS
<code>mcc_smoothed_risk</code>	1.805	0.476
<code>month_sin</code>	0.102	0.164
<code>is_risky_month</code>	0.061	0.111
<code>client_mcc_is_new</code>	0.029	0.034
나머지 피처	이상 없음	—

`mcc_smoothed_risk`

가장 강한 drift. MCC별 거래 비중이 바뀌면서 risk map이 환경과 괴리.

모델 구조 변경보다 **train+check 혼합 재추정(artifact 업데이트)** 이 1순위 대응.

`month_sin` / `is_risky_month`

데이터 기간 차이에 기인했을 가능성. 모델 구조 변경 불필요,

고위험 월 재선정 검토 후순위.

`client_mcc_is_new`

novelty 비율 증가. 고객 행동 패턴 변화 가능성이므로

추가 분석 후 반영 여부 판단.

피처 전체가 drift된 게 아니라 MCC 관련 피처에 집중
모델의 구조적 안정성은 확인됨,
운영 환경 적응은 artifact 업데이트로 충분.

Implication

① 제약이 설계를 결정한다

더 좋은 모델이 아니라 각 제약에 맞는 모델을 선택했다.

Latency 제약 → LR, Context 제약 → LGB. 모델 선택은 성능표가 아니라 문제 구조에서 나왔다.

② 피처는 가설에서 시작해 데이터로 끝난다

모든 피처는 사기범의 행동 가설에서 출발했다.

가설이 맞아도 데이터가 지지하지 않으면 DROP했고 직관과 달라도 데이터가 지지하면 KEEP했다.

EDA → 다중공선성 → SHAP → Ablation, 네 단계를 모두 통과한 피처만 남았다.

③ 모델의 끝은 배포가 아니라 운영이다

mcc_smoothed_risk에 강한 drift 확인. 모델 재학습 없이 artifact 업데이트만으로 대응 가능한 구조로 설계됐다.

Since we learned Tableau...



https://public.tableau.com/app/profile/51676566/viz/Monitoring_17716483699980/1?publish=yes

THANK YOU